



GENERALITAT
VALENCIANA

Conselleria d'Educació,
Universitats i Ocupació

ceedcv

CENTRE ESPECÍFIC
D'EDUCACIÓ A DISTÀNCIA DE
LA COMUNITAT VALENCIANA

Unitat 03 — Variables, tipus i operadors

Bloc 01

1r CFGS DAW · Programació

Sergi García Barea



Variables i tipus primitius

La idea en una frase

“

Les variables són **caixes etiquetades** en la memòria de l'ordinador, i Java et ofereix **8 tamanys de caixa** (els tipus primitius) perquè tries el que millor li va a cada dada.

En unitats anteriors, el teu programa només cridava text per la consola. Ara li donaràs memòria: guardarà la teua edat, el teu nom, la teua nota mitjana i fins i tot si tens gana. I per a triar bé la caixa de cada dada, primer has de conèixer el catàleg del magatzem.

La declaració: la recepta d'una caixa

Tipus · nom · valor

```
recepta Java
tipo nombreDeLaCaja = valorQueMetoDentro;
// Exemples reals
int edad = 25;           // caixa "edad" amb un 25
double precio = 19.99;   // caixa amb decimals
String nombre = "María"; // caixa que guarda text
boolean hambre = true;   // vertader/fals
```

1 Tipus — tamany i forma de la caixa

2 Nom — l'etiqueta de la caixa

3 Valor — el que fiques dins

i Les variables es diuen així perquè **varien**: `int edad = 25;` i el dia del teu aniversari, `edad = 26;`. L'etiqueta és la mateixa, el contingut canvia.

Les regles de nomenclatura

Les regles d'or de les etiquetes

- 1 Lletres, números, `_` i `$`. **Res d'espais** (evita ñ, ç, accents)
- 2 **No començar amb número:** `1numero` és il·legal; `numero1` és legal
- 3 **Les majúscules importen:** `edad`, `Edad` i `EDAD` són tres caixes distintes
- 4 **No usar paraules reservades:** `int`, `class`, `if`, `while` ...
- 5 **camelCase:** `miVariableEjemplo`

• • • noms

Java

```
// ✅ Correcte
int numeroAlumnos = 30;
double notaMedia = 7.5;

// ❌ Incorrecte
int 1numero = 30;    // comença per número
double nota media = 7.5; // espai en el nom
int class = 30;      // paraula reservada
```

Els 8 primitius: el catàleg de caixes

Tipus	Mida	El que cap	Analogia
byte	8 bits	-128 a 127	Caixa de llumins
short	16 bits	-32.768 a 32.767	Caixa de sabates
int	32 bits	± 2.147 milions	Caixa de mudança (la que més usaràs)
long	64 bits	± 9 trillions	Contenedor de vaixell
float	32 bits	Decimals precisió simple	Got d'aigua
double	64 bits	Decimals precisió doble	Cubell d'aigua
char	16 bits	Un sol caràcter Unicode	Una lletra en una caixa de sabates
boolean	1 bit	true o false	Interruptor de llum

i Nota: usa `int` per a quasi tot el numèric enter (només `long` si vas a contar estrelles) i `double` per a decimals, a menys que estalviar memòria siga el teu fetitxe.

Com es declara cada primitiu

els 8 primitius

Java

```
byte nivel = 100;
short poblacion = 30000;
int habitantes = 1500000; // el més usat
long distancia = 384400000L; // la L al final és obligatoria
float precio = 12.99f; // la f al final és obligatoria
double pi = 3.14159265359;
char letra = 'A'; // cometes SIMPLES per a char
boolean esJavaDivertido = true; // açò és opinable
```



`long` necessita la `L`, `float` la `f`, i `char` va amb cometes simples. Tres lletres que estalvien el 80% de les bronques amb el compilador.

Quina caixa use per a cada dada?

Triar la maleta del viatge



int

edats,
computadors,
puntuacions —
quasi tot enter



double

preus, notes
mitjanes,
temperatures —
qualsevol
decimal



boolean

respostes sí/no:
"ha aprovat?", "hi
ha connexió?"



char

una sola lletra:
'A', 'B', 'C'



long

nombres
astronòmics,
mil·lisegons, IDs
gegants



Quan dubtes entre `int` i `double` : *esta dada pot portar decimals?* Si sí → `double` . Si no → `int` . Simple.

Exercici: El guarda del magatzem

Quines declaracions compilen?



Eres el guarda d'un magatzem de dades. Et donen estes declaracions: **quines compilen i quines no?** Marca les que fallarien i per què.

Misteri.java

Java

```
int a = 150;           // ¿compila?  
int b = 10.5;          // ¿compila?  
double c = 7;          // ¿compila?  
char d = "A";          // ¿compila?  
boolean e = "true";    // ¿compila?  
long f = 30000000000L;  // ¿compila?  
int g = 3000000000;     // ¿compila?
```

Solució

- `a = 150` ✓ cap de sobres
- `b = 10.5` ✗ un int no admet decimals
- `c = 7` ✓ enter → double implícit
- `d = "A"` ✗ char usa cometes simples
- `e = "true"` ✗ amb cometes és text
- `f = 30000000000L` ✓ la L diu "això és un long"
- `g = 3000000000` ✗ 3.000M no cap en un int (tope 2.147M)

Mini-chequeig: primitius



Posat a prova en 30 segons:

1. Quin tamany de caixa usaríes per a guardar el nombre d'habitants de la Terra (més de 8.000 milions)?
2. Per què `char letra = "A";` no compila i `char letra = 'A';` sí?
3. Quina és la diferència entre `float` i `double` en una frase?
4. Per què `Edad`, `edad` i `EDAD` són tres variables distintes?



Respostes: 1) **long** — més de 2.147M no cap en un int · 2) char va amb **cometes simples** · 3) double té **doble precisió** (64 bits) · 4) les **majúscules importen**: cada variació és una caixa distinta.

String i constants

La idea en una frase

“

`String` és una **classe** (no un primitiu) que guarda text, és **immutable** com una foto, i `final` és el superglue que converteix qualsevol caixa en una **constant** que no es pot tocar.

strings

Java

```
String saludo = "Hola, DAW"; // forma normal  
String nombre = new String("Ana"); // constructor
```



`String` va amb **cometes dobles** `"..."` .
les simples `'...'` són només per a `char` (un
únic caràcter).

La immutabilitat i la trampa del ==

No toques, que es trenca

Immutables

Quan fas `texto = texto + " mundo"`, Java **no modifica** "Hola": tira el vell i crea un String nou. El text original queda congelat per sempre.

 Comparar Strings amb `=` pregunta "són el mateix objecte?", no "tenen el mateix text?".

 **Regla d'or:** els String sempre es comparen amb `.equals()`. Si uses `=`, tard o d'hora et mossegarà en un examen.

PoolTrampa.java

Java

```
String a = "Hello";
String b = "Hello";
String c = new String("Hello");
a = b      // true (mateix objecte)
a = c      // false (objecte nou)
a.equals(c) // true (mateix text)
```

Constants amb final

Caixes amb superglue

Constants.java

Java

```
final double IVA = 0.21;
final int MAXIMO_INTENTOS = 3;
final String NOMBRE_APP = "Gestión DAW";
IVA = 0.10; // ERROR: no pots reassignar!
```



Per convenció,
MAJÚSCULES_AMB_GUIONS: crida "¡SOC IMMUTABLE!" a qualsevol que lligue el codi.



Exemple guiat: la factura

Factura.java

Java

```
final double IVA = 0.21;
double precioBase = 50.0;
double ivaAplicado = precioBase * IVA;
double precioFinal = precioBase + ivaAplicado;
System.out.println("Total: " + precioFinal + "€");
// Total: 60.5€
```



Si l'IVA canvia demà, edites **una línia**, no les 50 on vas usar el 0.21.

Exercici: l'embolic de Strings

Què imprimeix?



Sense executar, digues què imprimeix exactament este codi.

Embollic.java

Java

```
String x = "Java";  
String y = "Java";  
String z = new String("Java");  
System.out.println(x == y);  
System.out.println(x == z);  
System.out.println(x.equals(z));
```

 **Solució:** true , false , true

- `x == y` → **true**: els dos literals comparteixen el pool de Strings
- `x == z` → **false**: `new` crea un objecte nou, sense referència compartida
- `x.equals(z)` → **true**: compara **text**, no referències

 Clàssic d'examen. Si l'encertes a la primera, la unitat la portes bé.

Operadors aritmètics

La idea en una frase

“

Els operadors aritmètics (+, -, *, /, %) són les màquines de pesos del gimnàs de dades: transformen les teues variables, i la **divisió entera** i la **precedència** són les trampes que separen els que saben dels que improvisen.

 + press de banca: $5 + 3 = 8$

 - curl de bíceps: $5 - 3 = 2$

 * sentadilla: $5 * 3 = 15$

 / pes mort: $10 / 3 = 3$ (enters) o $10.0 / 3 = 3.333 \dots$

 % l'abdominal odiat: $10 \% 3 = 1$ (el reste)

 Sense % no existiria res cíclic: parell/senar, torns, rellotges, jocs...

La divisió entera mata

Si divideixes dos enters, Java et retorna un enter

Divisio.java

Java

```
int alumnos = 17;
int grupos = 5;
System.out.println(alumnos / grupos);
// 3 - Java diu que cada grup té 3 alumnes
int a = 10, b = 3;
System.out.println(a / b); // 3 (entera)
System.out.println(a % b); // 1 (reste)
System.out.println((double) a / b); // 3.333...
```



5 / 2 és 2 · 5 / 2.0 és 2.5 · (double) 5 / 2 és 2.5. Memoritza-ho com un mantra.



Precedència: la llei del menjador

1

Parèntesis () — passe VIP, van els primers

2

*** / %** — els populars

3

+ - — els normals, els últims



Quan dubtes, **posa parèntesis**: no dolen i el teu jo del futur t'ho agrairà.

Assignació composta i increment

La drecera peresosa

⚡ Dreceres d'assignació

● ● ● Dreceres.java

Java

```
int x = 10;  
x += 5; // x = 15 (x = x + 5)  
x -= 3; // x = 12  
x *= 2; // x = 24  
x /= 4; // x = 6  
x %= 3; // x = 0
```

💪 ++ i --: dues cares

● ● ● Increments.java

Java

```
int a = 5;  
int b = a++; // POST: usa (5), puja → b=5, a=6  
int c = ++a; // PRE: puja, després usa → a=7, c=7
```



Si uses `++` dins d'una expressió complicada, escrius codi que ni tu entendràs en una setmana. Usa'ls **sols**, en la seua pròpia línia.

Exercici: l'acròbata de les variables

Sense executar, calcula quant val tot



Don Tip: desglossa l'expressió pas a pas. Quin valor té cada variable en cada moment? Anota-ho, no ho faces de memòria.

Acròbata.java

Java

```
int x = 3;
int y = x++ + ++x;
System.out.println("x = " + x + ", y = " + y);
```

Solució: `x = 5, y = 8`

1

`x = 3`

2

`x++` : usa 3, després puja x a 4

3

`++x` : puja x a 5 i usa 5

4

`y = 3 + 5 = 8, x = 5`

Relacionals, lògics i ternari

La idea en una frase

“

Els operadors **relacionals** comparen valors i retornen un boolean; els **lògics** combinen booleans; i el **ternari** resumix una decisió en una sola línia.

El jutge de la discussió

Els relacionals sempre retornen `true` o `false`: el jutge que dicta sentència sobre dos valors.

 **= no és ==.** = assigna ("guarda açò"); == compara ("són iguals?"). L'error més clàssic de la història de la programació.

Jutge.java

Java

```
int edad = 18;
edad ≥ 18 // true
edad ≠ 18 // false
edad > 21 // false
edad < 21 // true
```

Operadors relacionals

Taula del jutge (edad = 18)

Operador	Significat	Exemple (edad = 18)
<code>==</code>	Igual que	<code>edad == 18</code> → true
<code>!=</code>	Distint de	<code>edad != 18</code> → false
<code>></code>	Major que	<code>edad > 21</code> → false
<code><</code>	Menor que	<code>edad < 21</code> → true
<code>>=</code>	Major o igual	<code>edad ≥ 18</code> → true
<code><=</code>	Menor o igual	<code>edad ≤ 18</code> → true

i Tots retornen un `boolean` : el resultat d'una comparació és una dada que pots guardar, passar i combinar.

Lògics: el porter del club

&& (AND) · || (OR) · ! (NOT)

&& **AND:** més de 18 **I** entrada? Les dos condicions s'han de complir

|| **OR:** més de 18 **O** eres l'amo? Basta una

! **NOT:** nega la condició

Porter.java

Java

```
boolean entra = mayorEdad && tieneEntrada; // false
boolean entraVip = mayorEdad || tieneEntrada; // true
boolean noEsMenor = !(edad < 18); // true
```

⚡ Curtcircuit: el porter que no mira

Curtcircuit.java

Java

```
int x = 5;
boolean r = (x > 10) && (++x > 0);
// false, i x segueix sent 5: el ++x mai no s'executa
```

i Amb `&&`, si el primer és `false`, Java **ni mira** el segon. Amb `||`, si el primer és `true`, tampoc. A més et protegeix: `(algo != null) && algo.metodo()` evita el crash.

El ternari i l'exemple guiat

Un if-else de butxaca



El ternari

Ternari.java

Java

```
int nota = 7;
String r = nota ≥ 5 ? "Aprobado" : "Suspenso";
// condició ? valorSiTrue : valorSiFalse
```



Els ternaris es poden **encadenar** per a tres casos o més.



El porter del club

ClubNoche.java

Java

```
int edad = 19;
boolean entrada = true, esVip = false;
boolean puede = (edad ≥ 18) && (entrada || esVip);
String msg = puede ? "¡Pasa!" : "Fora de aquí";
int golesL = 2, golesV = 2;
String res = golesL > golesV ? "Gana local"
      : golesL < golesV ? "Gana visitante" : "Empate";
```

Exercici: el detectiu de booleans

Sense executar, calcula el valor de cada variable



Don Tip: en una expressió amb `&&` i `||`, pregunta sempre: *i si el primer ja decideix?* Eixe és el curtcircuit.

Detectiu.java

Java

```
int n = 10;
boolean a = (n > 5) && (n < 20);    // ¿?
boolean b = (n > 15) || (++n == 11); // ¿?
boolean c = !(n == 10) && (n % 2 == 0); // ¿?
System.out.println(a);
System.out.println(b);
System.out.println(c);
System.out.println(n);
```



Solució: true, true, false i 11

- **a:** $10 > 5$ i $10 < 20 \rightarrow$ true
- **b:** $10 > 15$ és false $\rightarrow$ Java avalua `++n == 11` : puja n a 11 i compara $\rightarrow$ true
- **c:** `!(n == 10)` amb $n=11 \rightarrow$!false $\rightarrow$ true; $\&\&$ amb $11 \% 2 == 0 \rightarrow$ false $\rightarrow$ false
- **n** acabà en **11** pel `++n` de la línia 2

Casting i conversions

La idea en una frase

“ El casting converteix un valor d'un tipus a un altre: la conversió **implícita** (widening) la fa Java sol i sense pèrdues, mentre que l'**explícita** (narrowing) la forces tu amb `(tipo)` i pots perdre dades pel camí.

Widening: mudar-se a caixa més gran

`byte → short → int → long → float → double`

Qualsevol tipus cap al de la seua dreta sense perdre ni un bit. Java somriu i et deixa.

Narrowing: la maleta XXL en un Smart

Java es nega. Has d'empènyer amb `(tipo)` : estàs dient *"confia en mi, sé el que faig"*.

 El casting sobre un double **trunca**, no redoniga: `(int) 19.99` dona 19. Per a redonir, `Math.round()` .

El truncament i el desbordament

El cèntim oblidat i l'elefant en el Mini Cooper

Truncament

Trunca.java

Java

```
double precio = 9.99;
int precioEntero = (int) precio;
System.out.println(precioEntero);
// 9 - t'acaben de llevar 0.99€
```

Desbordament

Desborda.java

Java

```
int grande = 300;
byte pequeno = (byte) grande;
System.out.println(pequeno);
// 44 - ¡quaranta-quatre!?
```

i 300 en binari és 100101100; un byte només guarda 8 bits → queda 00101100 = 44. Elefant al Mini, eix un gos salchicha.



El desbordament **no dona error**: Java no t'avisa i el programa segueix corrent amb un valor absurd. Comprova sempre que el valor cap abans de forçar el casting.

Exercici: seguix el guarda

Determina què imprimeix este programa

Cadena.java

Java

```
int a = 10;
double b = a;
int c = (int) b;
byte d = (byte) c;
System.out.println(d);
int grande = 300;
byte pequeno = (byte) grande;
System.out.println(pequeno);
```

 **Solució:** 10 i 44

- **Primer bloc:** cadena sense pèrdua 10 → 10.0 → 10 → 10. Imprimeix **10**
- **Segon bloc:** desbordament clàssic — 300 no cap en un byte, es truncaren els bits i queda **44**

✓ **Resum de la unitat (1/5):** variable = caixa etiquetada (tipus, nom, valor) · 8 primitius, tria `int` / `double` per defecte · String es compara amb `.equals()` · divisió entera trunca · el casting pot truncar i desbordar.

